

*Answer key***Problem 1.**

Convert each decimal number to hexadecimal:

$$26 = 1 \times 16 + 10 = 1A_{16}$$

$$60 = 3 \times 16 + 12 = 3C_{16}$$

$$175 = 10 \times 16 + 15 = AF_{16}$$

$$255 = 15 \times 16 + 15 = FF_{16}$$

**Problem 2.**

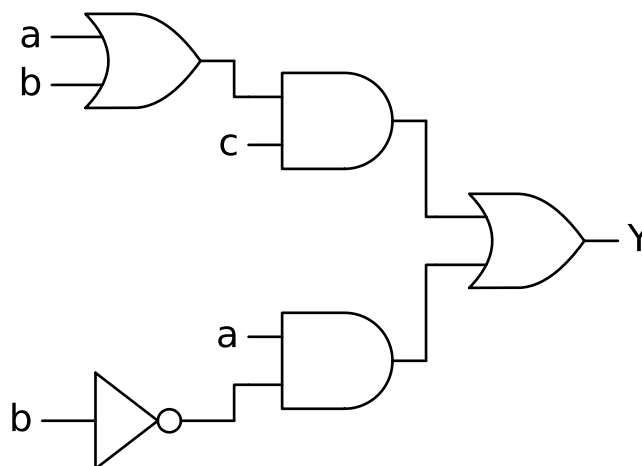
Convert each hexadecimal number to decimal:

$$1A_{16} = 1 \times 16 + 10 = 26$$

$$2F_{16} = 2 \times 16 + 15 = 47$$

$$B4_{16} = 11 \times 16 + 4 = 180$$

$$FF_{16} = 15 \times 16 + 15 = 255$$

**Problem 3.**

**Expression:**  $(A + B) \cdot C + A \cdot \overline{B}$

**Simplifies to:**  $B \cdot C + A \cdot \overline{B}$

**Truth table:**

| A | B | C | Y |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

---

#### Problem 4.

Convert each binary number to hexadecimal:

$$11010_2 = 0001\,1010 = 1A_{16}$$

$$101111_2 = 0010\,1111 = 2F_{16}$$

$$10110100_2 = 1011\,0100 = B4_{16}$$

$$1111_2 = 1111 = F_{16}$$

---

#### Problem 5.

Convert each hexadecimal number to binary:

$$3C_{16} = 3 \rightarrow 0011, C \rightarrow 1100 = 111100_2$$

$$A7_{16} = A \rightarrow 1010, 7 \rightarrow 0111 = 10100111_2$$

$$1E_{16} = 1 \rightarrow 0001, E \rightarrow 1110 = 11110_2$$

$$FF_{16} = F \rightarrow 1111, F \rightarrow 1111 = 11111111_2$$

---

### Problem 6.

Every device on a network has an *IP address*: four numbers separated by dots, each stored in one byte (so 0 to 255). Convert each part of these addresses to 8-bit binary.

192.168.1.1 = 11000000.10101000.00000001.00000001

10.0.0.255 = 00001010.00000000.00000000.11111111

8.8.8.8 = 00001000.00001000.00001000.00001000

Which one of these could *not* be a real IP address, and why?

192.168.0.1      10.0.256.4      172.16.0.255

**Answer:** 10.0.256.4 (256 is more than 255 — too big for one byte)

*Fun fact: 8.8.8.8 is one of Google's real public addresses.*

---

### Problem 7.

A black-and-white picture is a grid of *pixels*: each pixel is one bit — 1 means a filled (black) square, 0 means blank. Each row of 8 pixels is one byte, given below in hexadecimal. Convert each byte to 8-bit binary, then shade a square wherever there is a 1.

66<sub>16</sub> = 01100110

FF<sub>16</sub> = 11111111

FF<sub>16</sub> = 11111111

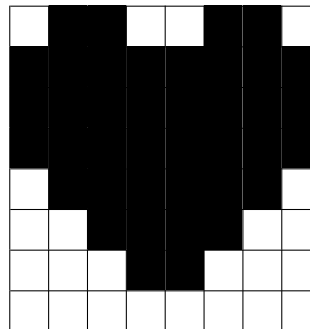
FF<sub>16</sub> = 11111111

7E<sub>16</sub> = 01111110

3C<sub>16</sub> = 00111100

18<sub>16</sub> = 00011000

00<sub>16</sub> = 00000000



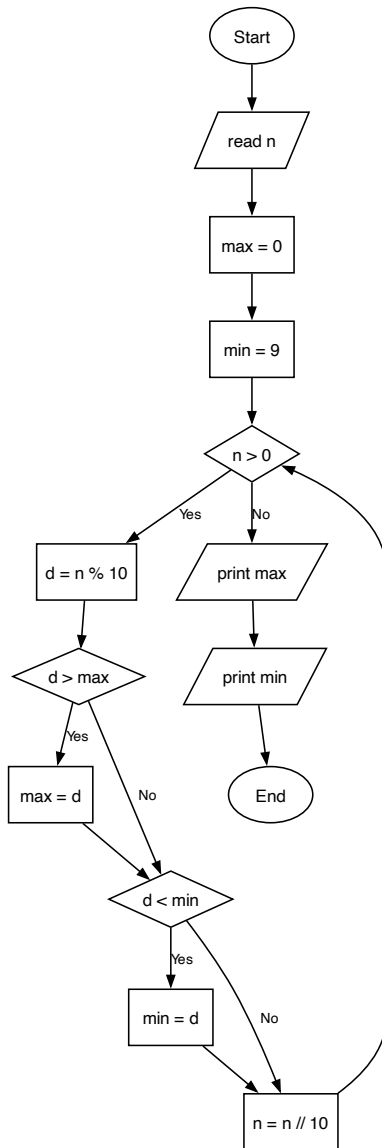
*What picture appears? The whole image is just 8 bytes — 64 ones and zeros. Roughly how many bytes would a picture twice as wide and twice as tall (16 × 16) need?*

**The picture:** a heart

---

### Problem 8.

Trace this algorithm for  $n = 5184$ , and complete the trace table.



| n    | max | min | d | output |
|------|-----|-----|---|--------|
| 5184 |     |     |   |        |
| 5184 | 0   |     |   |        |
| 5184 | 0   | 9   |   |        |
| 5184 | 0   | 9   | 4 |        |
| 5184 | 4   | 9   | 4 |        |
| 5184 | 4   | 4   | 4 |        |
| 518  | 4   | 4   | 4 |        |
| 518  | 4   | 4   | 8 |        |
| 518  | 8   | 4   | 8 |        |
| 51   | 8   | 4   | 8 |        |
| 51   | 8   | 4   | 1 |        |
| 51   | 8   | 1   | 1 |        |
| 5    | 8   | 1   | 1 |        |
| 5    | 8   | 1   | 5 |        |
| 0    | 8   | 1   | 5 |        |
| 0    | 8   | 1   | 5 | 8      |
| 0    | 8   | 1   | 5 | 1      |

**Output:** 8, 1